

Spring Boot 미니 게시판

기능 보완 실습 및 핵심 용어 정리

H2 REST API + JavaScript fetch 기반 게시판 프로젝트 보충 자료

수업일: 2026년 8월 2일

이 자료는 기존 미니 게시판 CRUD 구현 이후, 날짜·조회수·좋아요·검색 기능을 추가하고 프로젝트에서 사용한 주요 용어를 정리하기 위한 수업용 자료입니다.

목차

1부. 기능 보완 실습 8개

1. 공백 입력 방지 보완
2. 작성 날짜 자동 저장
3. 최신 게시물 먼저 표시
4. 목록에 날짜 표시
5. 게시물 개수 표시
6. 조회수 기능
7. 좋아요 기능
8. 제목 검색과 글자 수 표시

2부. 프로젝트 핵심 용어 정리

3부. 자주 나오는 오류와 확인 순서

4부. 질문과 답

수업 진행 권장 순서: 날짜 -> 최신순 -> 개수 -> 조회수 -> 좋아요 -> 검색 순으로 진행하면, 학생들이 기능이 추가되는 과정을 화면에서 바로 확인할 수 있습니다.

1부. 기능 보완 실습 8개

이미 완성한 게시판에 작은 기능을 하나씩 추가한다. 모든 기능은 화면에서 바로 결과를 확인할 수 있도록 구성한다.

번호	기능	수정 위치	확인 방법
1	공백 입력 방지 보완	write.html	스페이스만 입력 후 등록 시도
2	작성 날짜 자동 저장	Board.java	새 글 등록 후 DB 확인
3	최신 게시글 먼저 표시	BoardRepository.java, Controller	새 글이 목록 위에 나오는지 확인
4	목록에 날짜 표시	list.html	목록에서 작성일 확인
5	게시글 개수 표시	list.html	목록 제목 옆 개수 확인
6	조회수 기능	Board.java, Controller, detail.html	상세 페이지 새로고침 후 숫자 증가 확인
7	좋아요 기능	Board.java, Controller, detail.html	좋아요 버튼 클릭 후 숫자 증가 확인
8	제목 검색과 글자 수 표시	list.html, write.html	검색어 입력 및 글자 수 변화 확인

1. 공백 입력 방지 보완

빈칸 검사를 이미 했다면, 공백만 입력하는 경우까지 막는다. 입력값의 앞뒤 공백을 제거한 뒤 검사한다.

```
const title = document.getElementById("title").value.trim();
const writer = document.getElementById("writer").value.trim();
const content = document.getElementById("content").value.trim();

if (!title || !writer || !content) {
    alert("제목, 작성자, 내용을 모두 입력해 주세요.");
    return;
}
```

trim()은 문자열 앞뒤의 공백을 제거한다. " "처럼 스페이스만 입력한 값도 빈 문자열로 처리할 수 있다.

2. 작성 날짜 자동 저장

게시글을 새로 저장할 때 현재 날짜와 시간을 자동으로 넣는다.

Board.java - import 추가

```
import java.time.LocalDateTime;
import jakarta.persistence.PrePersist;
```

Board.java - 필드와 메서드 추가

```
private LocalDateTime createdAt;

@PrePersist
public void createDate() {
    createdAt = LocalDateTime.now();
}
```

@PrePersist는 Entity가 DB에 처음 저장되기 직전에 실행된다. 새 글을 등록할 때 createdAt에 현재 시간이 자동으로 들어간다.

3. 최신 게시물 먼저 표시

id가 큰 게시물일수록 나중에 작성된 글이다. id를 내림차순으로 정렬하면 최신 글이 위에 표시된다.

BoardRepository.java

```
import java.util.List;

public interface BoardRepository extends JpaRepository<Board, Long> {

    List<Board> findAllByIdDesc();
}
```

BoardApiController.java

```
@GetMapping
public List<Board> list() {
    return boardRepository.findAllByIdDesc();
}
```

findAllByIdDesc()는 메서드 이름만 보고 Spring Data JPA가 쿼리를 만들어 주는 방식이다.

4. 목록에 날짜 표시

서버에서 받은 createdAt 값을 JavaScript에서 보기 좋은 날짜 형식으로 변환한다.

list.html - 표 제목

```
<thead>
  <tr>
    <th>게시일</th>
    <th>번호</th>
    <th>제목</th>
    <th>작성자</th>
  </tr>
</thead>
```

list.html - 목록 출력

```
data.forEach(board => {
    const row = document.createElement("tr");

    const date = board.createdAt
    ? new Date(board.createdAt).toLocaleDateString()
    : "날짜 없음";

    row.innerHTML = `
    <td>${date}</td>
    <td>${board.id}</td>
    <td onclick="goDetail(${board.id})"
      style="cursor:pointer; color:green; width:40%"
      >${board.title}
    </td>
    <td>${board.writer}</td>
  `;

    list.appendChild(row);
});
```

주의: board 변수는 data.forEach(board => { ... }) 안에서만 사용할 수 있다. forEach 바깥에서 board.createdAt을 사용하면 board is not defined 오류가 난다.

5. 게시물 개수 표시

목록에 현재 게시물 개수를 함께 표시한다. 기능이 짧고 결과가 바로 보여 중간 확인용으로 좋다.

list.html - 제목 부분

```
<h1>
  게시판 목록 <span id="board-count"></span>
</h1>
```

list.html - 데이터를 받은 뒤

```
document.getElementById("board-count").textContent
= `${data.length}개`;
```

data.length는 서버에서 받은 게시물 배열의 개수를 의미한다.

6. 조회수 기능

상세 페이지를 열 때마다 조회수를 1 증가시킨다. 조회 -> 값 변경 -> 다시 저장 흐름을 확인할 수 있다.

Board.java

```
private int viewCount;
```

BoardApiController.java - 기존 상세 조회 메서드 수정

```
@GetMapping("/{id}")
public Board detail(@PathVariable Long id) {
    Board board = boardRepository.findById(id).orElseThrow();

    board.setViewCount(board.getViewCount() + 1);

    return boardRepository.save(board);
}
```

detail.html

```
<p>조회수: <span id="viewCount"></span></p>
```

```
document.getElementById("viewCount").textContent
= board.viewCount;
```

@GetMapping("/{id}") 메서드를 새로 하나 더 만들지 않는다. 기존 상세 조회 메서드가 있으면 그 메서드를 수정한다. 같은 주소의 GET 메서드가 2개 있으면 오류가 난다.

7. 좋아요 기능

좋아요 버튼을 누를 때마다 좋아요 수를 1 증가시킨다. 로그인 기능이 없으므로 같은 사람이 여러 번 누르는 것은 막지 않는다.

Board.java

```
private int likeCount;
```

BoardApiController.java

```
@PutMapping("/{id}/like")
public Board like(@PathVariable Long id) {
    Board board = boardRepository.findById(id).orElseThrow();

    board.setLikeCount(board.getLikeCount() + 1);

    return boardRepository.save(board);
}
```

detail.html - 버튼

```
<button onclick="likeBoard()">
    좋아요 <span id="likeCount">0</span>
</button>
```

detail.html - JavaScript

```
function likeBoard() {
    fetch("/api/boards/" + id + "/like", {
        method: "PUT"
    })
    .then(response => response.json())
    .then(board => {
        document.getElementById("likeCount").textContent
            = board.likeCount;
    });
}
```

상세 조회 후 좋아요 수 표시

```
document.getElementById("likeCount").textContent
    = board.likeCount;
```

버튼 클릭 -> PUT 요청 -> 기존 좋아요 수에 1 추가 -> DB 저장 -> 화면 숫자 변경

8. 제목 검색과 글자 수 표시

8-1. 제목 검색

목록을 새로 불러오지 않고, 이미 받은 게시글 배열에서 제목에 키워드가 포함된 글만 골라 보여준다.

```
<input type="text" id="keyword" placeholder="제목 검색">
<button onclick="searchBoard()">검색</button>

let allBoards = [];

fetch("/api/boards")
  .then(response => response.json())
  .then(data => {
    allBoards = data;
    printBoards(data);
  });

function searchBoard() {
  const keyword = document.getElementById("keyword").value.trim();

  const result = allBoards.filter(board =>
    board.title.includes(keyword)
  );

  printBoards(result);
}
```

8-2. 내용 글자 수 표시

글을 입력할 때마다 현재 글자 수를 표시한다.

```
<textarea id="content" oninput="countText()"></textarea>
<p><span id="text-count">0</span> / 1000자</p>

function countText() {
  const content = document.getElementById("content").value;

  document.getElementById("text-count").textContent
    = content.length;
}
```

검색은 빠른 학생용 선택 실습으로 돌려도 좋다. 글자 수 표시는 코드가 짧아 전체 학생이 따라 하기 쉽다.

기능 보완 확인표

확인	항목
☐	스페이스만 입력한 게시글이 등록되지 않는다.
☐	새 게시글을 등록하면 작성 날짜가 저장된다.
☐	최신 게시글이 목록 위쪽에 나온다.
☐	목록에서 작성 날짜를 확인할 수 있다.
☐	게시글 개수가 목록 제목 옆에 표시된다.
☐	상세 페이지를 열면 조회수가 증가한다.
☐	좋아요 버튼을 누르면 좋아요 수가 증가한다.
☐	제목 검색 또는 글자 수 표시가 동작한다.
☐	Console에 빨간 오류가 없다.
☐	H2 Console 또는 phpMyAdmin에서 데이터가 확인된다.

2부. 프로젝트 핵심 용어 정리

용어를 외우기보다, 프로젝트 안에서 어디에 쓰였는지 연결해서 이해한다.

전체 흐름

용어	직관적 설명	이 프로젝트에서의 위치
Spring Boot	Java 웹 서버를 빠르게 만들 수 있게 도와주는 도구이다.	이 프로젝트에서는 게시글 API 서버를 실행한다.
서버	요청을 받아 처리하고 결과를 돌려주는 프로그램이다.	Spring Boot가 서버 역할을 한다.
클라이언트	서버에 요청을 보내는 쪽이다.	브라우저, JavaScript fetch(), Thunder Client가 해당 된다.
Request	클라이언트가 서버에 보내는 요청이다.	게시글 목록 조회, 등록, 수정, 삭제 요청이 있다.
Response	서버가 요청 처리 후 돌려주는 응답이다.	Board 객체나 게시글 목록이 JSON으로 돌아온다.
localhost	내 컴퓨터 자신을 가리키는 주소이다.	http://localhost:8080 으로 내 컴퓨터의 서버에 접속한다.
port	한 컴퓨터 안에서 프로그램을 구분하는 번호이다.	Spring Boot 기본 포트는 8080이다.

Controller와 API

용어	직관적 설명	이 프로젝트에서의 위치
Controller	브라우저 요청을 처음 받는 Java 클래스이다.	BoardApiController가 게시글 요청을 받는다.
@RestController	이 클래스가 REST API 응답을 처리한다고 표시한다.	Java 객체를 JSON으로 변환해 응답한다.
@RequestMapping	Controller의 기본 주소를 정한다.	/api/boards가 게시글 API의 기본 주소이다.
Mapping	주소와 메서드를 연결하는 것이다.	@GetMapping, @PostMapping 등이 있다.
Endpoint	요청 방식과 주소가 합쳐진 하나의 API 지점이다.	GET /api/boards 와 POST /api/boards는 서로 다른 Endpoint이다.
REST API	주소와 HTTP 방식으로 데이터를 처리하는 API 구조이다.	/api/boards에 GET, POST, PUT, DELETE를 사용한다.
JSON	프로그램끼리 데이터를 주고받기 위한 문자열 형식이다.	게시글 데이터가 JSON으로 이동한다.

HTTP와 CRUD

용어	직관적 설명	이 프로젝트에서의 위치
HTTP	브라우저와 서버가 통신할 때 사용하는 약속이다.	요청 주소, 요청 방식, 응답 상태 코드가 포함된다.
GET	데이터를 조회할 때 사용한다.	게시글 목록과 상세 조회에 사용한다.
POST	새 데이터를 등록할 때 사용한다.	새 게시글 등록에 사용한다.
PUT	기존 데이터를 수정할 때 사용한다.	게시글 수정, 좋아요 증가에 사용할 수 있다.
DELETE	데이터를 삭제할 때 사용한다.	게시글 삭제에 사용한다.
CRUD	등록, 조회, 수정, 삭제 기능을 묶어 부르는 말이다.	게시판의 기본 기능 전체를 의미한다.
Status Code	서버 처리 결과를 숫자로 알려주는 값이다.	200, 400, 404, 500 등을 확인한다.

Entity와 DB

용어	직관적 설명	이 프로젝트에서의 위치
Entity	DB 테이블과 연결되는 Java 클래스이다.	Board 클래스가 board 테이블과 연결된다.
Field	클래스 안에 선언된 데이터이다.	title, writer, content, createdAt 등이 필드이다.
Table	DB에서 데이터를 저장하는 표이다.	board 테이블에 게시글이 저장된다.
Column	테이블의 세로 항목이다.	id, title, writer, content 컬럼이 있다.
Row	테이블에 저장된 한 줄 데이터이다.	게시글 하나가 board 테이블의 한 행이다.
Primary Key	각 데이터를 구분하는 고유한 값이다.	게시글 id가 기본키이다.
@Entity	이 클래스를 DB 테이블로 사용한다고 표시한다.	Board 클래스 위에 붙인다.
@Id	기본키 필드를 지정한다.	Board의 id 필드에 붙인다.
@GeneratedValue	id 값을 자동으로 만들게 한다.	POST 요청에 id를 넣지 않는 이유이다.
@Column	컬럼 길이 같은 세부 설정을 지정한다.	content 길이를 1000자로 지정할 수 있다.

JPA와 Repository

용어	직관적 설명	이 프로젝트에서의 위치
JPA	Java 객체로 DB를 다룰 수 있게 해 주는 표준 규칙이다.	save(), findAll() 같은 방식으로 DB 작업을 한다.
ORM	객체와 DB 테이블을 연결하는 방식이다.	Board 객체와 board 테이블이 연결된다.
Hibernate	JPA를 실제로 실행해 주는 대표 구현체이다.	SQL을 대신 만들어 DB에 전달한다.
Repository	DB 작업을 맡는 인터페이스이다.	BoardRepository가 게시글 저장과 조회를 담당한다.
JpaRepository	기본 DB 기능을 제공하는 인터페이스이다.	save(), findAll(), findById(), deleteById()를 제공한다.
extends	기존 기능을 상속받는다는 뜻이다.	BoardRepository가 JpaRepository 기능을 물려받는다.
save()	새 데이터를 저장하거나 기존 데이터를 수정한다.	게시글 등록과 수정에 사용된다.
findAll()	전체 데이터를 조회한다.	게시글 목록 조회에 사용된다.
findById()	id로 데이터 하나를 찾는다.	상세 조회, 수정, 조회수 증가에 사용된다.
deleteById()	id에 해당하는 데이터를 삭제한다.	게시글 삭제에 사용된다.

쿼리 메서드와 정렬

용어	직관적 설명	이 프로젝트에서의 위치
Query	DB에 데이터를 조회하거나 변경하라고 내리는 명령이다.	SELECT, INSERT, UPDATE, DELETE가 있다.
findAllByOrderByDesc()	전체 데이터를 id 내림차순으로 조회하는 JPA 메서드이다.	최신 게시글을 먼저 보여줄 때 사용한다.
ASC	오름차순이다. 작은 값부터 큰 값 순서이다.	1, 2, 3, 4 순서이다.
DESC	내림차순이다. 큰 값부터 작은 값 순서이다.	4, 3, 2, 1 순서이다.
Optional	값이 있을 수도 없을 수도 있음을 표현한다.	findById() 결과로 자주 만난다.
orElseThrow()	값이 있으면 꺼내고 없으면 오류를 발생시킨다.	없는 id를 조회할 때 예외가 난다.

날짜와 시간

용어	직관적 설명	이 프로젝트에서의 위치
LocalDateTime	날짜와 시간을 함께 저장하는 Java 자료형이다.	createdAt 필드에 사용한다.
LocalDateTime.now()	현재 날짜와 시간을 가져온다.	게시글 저장 시 작성 시간을 넣는다.
createdAt	데이터가 생성된 시점을 의미하는 필드명이다.	게시글 작성일을 저장한다.
@PrePersist	Entity가 처음 저장되기 직전에 실행할 메서드를 지정한다.	createdAt 자동 입력에 사용한다.
toLocaleDateString()	날짜를 사용자의 지역 형식에 맞게 표시한다.	목록에서 작성일을 보기 좋게 보여준다.
toLocaleString()	날짜와 시간을 함께 보기 좋은 형식으로 표시한다.	상세 화면에서 작성 시간을 보여줄 때 쓸 수 있다.

설정과 DB

용어	직관적 설명	이 프로젝트에서의 위치
application.properties	Spring Boot 설정을 작성하는 파일이다.	포트, DB 연결, JPA 설정을 작성한다.
server.port	서버가 사용할 포트 번호를 정한다.	server.port=8080처럼 사용한다.
DataSource	애플리케이션이 접속할 DB 정보를 말한다.	url, username, password가 필요하다.
JDBC	Java가 DB에 연결할 때 사용하는 표준 방식이다.	jdbc:h2:file:./testdb 같은 주소를 사용한다.
H2	학습용으로 많이 쓰는 가벼운 DB이다.	로컬 실습에서 사용한다.
MySQL	실제 서비스에서도 많이 쓰는 DB이다.	배포 DB로 사용할 수 있다.
H2 Console	브라우저에서 H2 데이터를 확인하는 화면이다.	/h2-console로 접속한다.
phpMyAdmin	브라우저에서 MySQL을 확인하고 관리하는 도구이다.	Clever Cloud DB 데이터 확인에 사용한다.
ddl-auto	Entity를 기준으로 DB 테이블을 어떻게 관리할지 정한다.	create, update, none 등을 설정한다.
create	서버 시작 시 테이블을 새로 만든다.	기존 데이터가 사라질 수 있다.
update	기존 데이터를 유지하면서 필요한 구조를 반영한다.	수업용 프로젝트에서 가장 무난하다.
none	Hibernate가 DB 구조를 건드리지 않는다.	테이블을 직접 관리할 때 사용한다.

HTML, CSS, JavaScript

용어	직관적 설명	이 프로젝트에서의 위치
HTML	웹페이지의 구조를 만드는 언어이다.	제목, 표, 버튼, 입력창을 만든다.
CSS	웹페이지의 디자인을 담당한다.	배경색, 버튼 모양, 여백을 바꾼다.
JavaScript	웹페이지의 동작을 처리한다.	버튼 클릭, 서버 요청, 화면 변경을 처리한다.
static 폴더	브라우저에 그대로 전달되는 파일을 넣는 위치이다.	list.html, write.html, detail.html이 들어간다.
id 속성	HTML 요소 하나를 찾기 위한 이름이다.	document.getElementById()로 찾는다.
class 속성	여러 요소에 같은 CSS를 적용하기 위한 이름이다.	.btn 같은 선택자로 사용한다.
DOM	HTML 문서를 JavaScript에서 다룰 수 있게 만든 구조이다.	입력값 읽기, 표 추가 등에 사용된다.
document	현재 웹페이지 전체를 나타내는 객체이다.	document.getElementById()로 요소를 찾는다.
innerHTML	HTML 요소 안에 새로운 HTML을 넣는다.	게시글 목록의 tr, td를 만들 때 사용한다.
textContent	HTML 요소 안의 글자만 변경한다.	조회수나 좋아요 숫자 표시 때 사용한다.

fetch와 화면 처리

용어	직관적 설명	이 프로젝트에서의 위치
fetch()	JavaScript에서 서버로 HTTP 요청을 보내는 기능이다.	목록 조회, 등록, 수정, 삭제에 사용한다.
상대 주소	현재 접속한 서버를 기준으로 요청하는 주소이다.	fetch("/api/boards")처럼 쓴다.
절대 주소	전체 주소를 직접 쓰는 방식이다.	배포 후 localhost가 남아 있으면 문제가 될 수 있다.
headers	요청에 대한 부가 정보를 보낸다.	Content-Type을 application/json으로 지정한다.
Content-Type	보내는 데이터 형식을 알려준다.	JSON을 보낼 때 application/json을 사용한다.
JSON.stringify()	JavaScript 객체를 JSON 문자열로 바꾼다.	POST, PUT 요청 body에 넣는다.
response.json()	서버 응답 JSON을 JavaScript 객체로 바꾼다.	then 안에서 데이터를 사용할 수 있게 한다.
Promise	나중에 결과가 도착하는 작업을 표현한다.	fetch() 요청은 응답이 나중에 온다.
.then()	앞 작업이 성공한 뒤 실행할 코드를 작성한다.	서버 응답을 받은 뒤 화면을 그린다.
.catch()	오류가 났을 때 실행할 코드를 작성한다.	요청 실패 메시지를 표시한다.

배열, 반복, 검색

용어	직관적 설명	이 프로젝트에서의 위치
Array	여러 데이터를 순서대로 담는 자료구조이다.	게시글 목록 data가 배열이다.
forEach()	배열의 데이터를 하나씩 반복 처리한다.	게시글 하나마다 tr을 만든다.
Scope	변수를 사용할 수 있는 범위이다.	forEach 안의 board는 그 안에서만 사용한다.
filter()	조건에 맞는 데이터만 골라 새 배열을 만든다.	제목 검색에서 사용한다.
includes()	문자열에 특정 글자가 포함되어 있는지 확인한다.	제목에 검색어가 있는지 확인한다.
length	문자열 길이나 배열 개수를 알려준다.	글자 수와 게시글 개수에 사용한다.
trim()	문자열 앞뒤 공백을 제거한다.	공백만 입력한 등록을 막는 데 사용한다.
Template Literal	백틱 문자열 안에 변수와 여러 줄 HTML을 넣는 방식이다.	row.innerHTML을 만들 때 사용한다.

화면 이동과 사용자 확인

용어	직관적 설명	이 프로젝트에서의 위치
location.href	다른 페이지로 이동시킨다.	목록, 상세, 글쓰기 페이지 이동에 사용한다.
Query String	주소 뒤에 값을 붙여 전달하는 방식이다.	detail.html?id=3 형태로 id를 전달한다.
URLSearchParams	주소의 Query String 값을 읽는다.	상세 페이지에서 id를 꺼낸다.
alert()	사용자에게 안내 메시지를 보여준다.	등록 완료, 오류 안내에 사용한다.
confirm()	확인과 취소 버튼이 있는 창을 보여준다.	삭제 전 확인창에 사용한다.
return	함수 실행을 즉시 끝낸다.	취소를 누르면 삭제 요청을 보내지 않는다.

개발자도구와 오류

용어	직관적 설명	이 프로젝트에서의 위치
Developer Tools	브라우저에서 오류와 요청을 확인하는 도구이다.	F12로 연다.
Console	JavaScript 오류와 로그를 확인하는 탭이다.	ReferenceError, TypeError 등을 확인한다.
Network	브라우저가 보낸 요청과 서버 응답을 확인하는 탭이다.	Status, Payload, Response를 본다.
Fetch/XHR	fetch() 같은 API 요청만 모아 보는 필터이다.	등록, 수정, 삭제 요청 확인에 좋다.
Payload	브라우저가 서버로 보낸 데이터이다.	POST 요청의 title, writer, content를 확인한다.
Response	서버가 브라우저로 돌려준 데이터이다.	오류 내용이나 JSON 결과를 확인한다.
ReferenceError	없는 변수나 함수를 사용했을 때 발생한다.	board is not defined가 대표 예이다.
TypeError	값의 종류가 예상과 다를 때 발생한다.	null.value 같은 코드에서 날 수 있다.
SyntaxError	문법이 잘못됐을 때 발생한다.	괄호, 따옴표, 중괄호 누락을 확인한다.
cannot find symbol	Java가 사용한 이름을 찾지 못했다는 컴파일 오류이다.	변수 선언 누락, 오타, import 누락을 확인한다.
BUILD FAILED	빌드 중 오류가 나서 실행되지 못했다는 뜻이다.	위쪽의 첫 번째 실제 오류를 먼저 본다.

빌드와 배포

용어	직관적 설명	이 프로젝트에서의 위치
Gradle	Java 프로젝트의 빌드와 라이브러리 관리를 담당한다.	bootRun으로 서버를 실행한다.
build.gradle	프로젝트가 사용할 라이브러리와 설정을 적는 파일이다.	Spring Web, JPA, H2 의존성이 들어간다.
Dependency	프로젝트가 사용하는 외부 라이브러리이다.	Spring Web, Lombok, MySQL Driver 등이 있다.
Gradle Wrapper	프로젝트에 포함된 Gradle 실행 도구이다.	.₩gradlew bootRun으로 실행한다.
Package	Java 클래스를 정리하는 이름 체계이다.	com.example.hello 같은 형태이다.
import	다른 패키지의 클래스를 현재 파일에서 사용할 수 있게 한다.	List, LocalDateTime 등을 import한다.
Deploy	내 컴퓨터에서 만든 프로그램을 인터넷 서버에 올리는 작업이다.	Render 같은 서비스에 배포한다.
Local	내 컴퓨터에서 실행하는 환경이다.	localhost:8080으로 확인한다.
Production	실제 사용자가 접속하는 배포 환경이다.	배포 URL로 접속한다.
Environment Variable	비밀번호 같은 값을 코드에 직접 쓰지 않고 배포 환경에 따로 넣는 설정이다.	DB_URL, DB_USERNAME, DB_PASSWORD에 사용한다.
Log	서버가 실행되며 남기는 기록이다.	500 오류는 배포 로그에서 원인을 찾는다.

3부. 자주 나오는 오류와 확인 순서

1. 화면은 뜨지만 버튼이 작동하지 않을 때

확인 위치	불 내용
Console	빨간 오류가 있는지 확인한다.
오류 줄 번호	list.html:99처럼 표시된 파일과 줄을 확인한다.
document.getElementById	document. 누락이나 id 오타를 확인한다.
함수 이름	onclick에 적은 함수 이름과 실제 함수 이름이 같은지 확인한다.

2. 등록, 수정, 삭제 요청이 실패할 때

상태 코드	의미	확인 위치
200	성공	Response와 화면 결과 확인
400	보낸 데이터 문제	Payload와 JSON 형식 확인
404	주소 또는 데이터 없음	fetch 주소와 Controller 매핑 확인
405	요청 방식 불일치	GET, POST, PUT, DELETE 확인
500	서버 내부 오류	Spring Boot 로그 또는 배포 로그 확인

3. DB가 계속 리셋될 때

application.properties에서 ddl-auto 값을 확인한다.

```
spring.jpa.hibernate.ddl-auto=create
```

위 설정은 서버 실행 시 테이블을 새로 만들 수 있다. 데이터 유지가 필요하다면 다음처럼 변경한다.

```
spring.jpa.hibernate.ddl-auto=update
```

설정	의미
create	실행할 때 테이블을 새로 만들 수 있어 기존 데이터가 사라질 수 있다.
update	기존 데이터를 유지하면서 필요한 테이블이나 컬럼을 반영한다.
none	Hibernate가 DB 구조를 건드리지 않는다.

4. 오류 확인 기본 순서

순서	확인
1	화면 문제인지, 요청 문제인지 먼저 구분한다.
2	Console에서 JavaScript 오류를 확인한다.
3	Network에서 요청 주소, 방식, 상태 코드를 확인한다.
4	Payload에서 서버로 보낸 값을 확인한다.
5	500 오류는 Spring Boot 로그 또는 Render 로그를 확인한다.
6	저장 여부는 H2 Console 또는 phpMyAdmin에서 확인한다.

4부. 질문과 답

질문	답
등록 요청은 어느 Controller가 받나요?	BoardApiController의 @PostMapping 메서드가 받습니다.
DB에 저장하는 코드는 어디인가요?	boardRepository.save(board)입니다.
findAll()은 누가 제공하나요?	BoardRepository가 상속한 JpaRepository가 제공합니다.
id를 POST에 넣지 않은 이유는 무엇인가요?	DB가 @GeneratedValue 설정에 따라 id를 자동 생성하기 때문입니다.
화면 문구는 어느 파일에서 바꾸나요?	src/main/resources/static 폴더 안의 HTML 파일에서 바꿉니다.
서버가 꺼지면 왜 요청이 실패하나요?	Controller가 브라우저 요청을 받을 수 없기 때문입니다.
조회수는 어느 순간 증가하나요?	상세 조회 API가 실행되어 게시글을 찾은 뒤 viewCount를 1 증가시킬 때 증가합니다.
좋아요는 왜 PUT 요청을 사용하나요?	기존 게시글의 likeCount 값을 변경하는 기능이기 때문입니다.
날짜가 기존 글에는 안 보이는 이유는 무엇인가요?	createdAt 필드를 추가하기 전에 저장된 기존 데이터에는 날짜 값이 없을 수 있기 때문입니다.
create와 update의 차이는 무엇인가요?	create는 테이블을 새로 만들 수 있고, update는 기존 데이터를 유지하면서 구조를 반영합니다.

전체 흐름 한 문장 정리

브라우저가 fetch()로 요청하면 Controller가 받고, Repository와 JPA가 DB 작업을 처리한 뒤, 결과를 JSON으로 돌려주면 JavaScript가 화면에 표시한다.

수업 마무리 확인: 학생이 자신의 사이트에서 등록, 목록, 상세, 수정, 삭제, 날짜, 조회수 또는 좋아요 중 하나를 직접 시연하고 관련 파일을 설명할 수 있으면 된다.